

Booking Backend

Frontend Developer Guide — API Endpoints & Data Model

Covers both properties on the shared backend:
Centurion Hotel & Mum's Garden Resort

One backend · one database · `property_id`-scoped | July 2026

Contents

1	How this backend is organized
2	Core data model, with example JSON for every entity
3	Standard response & error shape
4	Public API — content, availability, booking, payment
5	Guest account API (optional)
6	Booking + payment flow, step by step
7	Admin panel API
8	Roles & permissions cheat sheet
9	Things still pending client sign-off

1. How this backend is organized

There is **one** backend, **one** database, and **one** set of API routes shared by both properties. Nothing is duplicated per property — instead, almost every URL includes a `propertySlug`, and almost every response row carries a `property_id`. As a frontend dev, the mental model is simple:

```
/api/v1/properties/centurion-hotel/room-types
/api/v1/properties/mums-garden-resort/room-types
```

Same endpoint, same shape, different slug in the URL. You will build one set of components/services and pass the property slug in as a variable (e.g. from an env var, subdomain, or route param) — you should never need "if property == mums-garden" branching logic in the frontend for anything the API already returns.

Property	Slug	Example <code>property_id</code> (UUID)
Centurion Hotel CENTURION	<code>centurion-hotel</code>	<code>b3ac1eab-5b2d-4e2a-9c1a-111111111111</code>
Mum's Garden Resort MUMS GARDEN	<code>mums-garden-resort</code>	<code>d7f2c9a4-8e31-4b7a-9f2e-222222222222</code>

Room types differ a lot between the two properties (different amenities, bed types, naming). Anything property-specific that doesn't fit a fixed column lives in a free-form `attributes` object on the room type — treat it as an open bag of key/value data, not a fixed schema, when rendering room detail pages.

2. Core data model

This is the shape of the data you'll actually receive from the API. It's simplified from the full database schema — internal-only fields (audit logs, raw payment callbacks, etc.) are left out because the frontend never touches them directly.

2.1 Property

```
{
  "id": "d7f2c9a4-8e31-4b7a-9f2e-222222222222",
  "slug": "mums-garden-resort",
  "name": "Mum's Garden Resort",
  "timezone": "Africa/Nairobi",
  "currency": "KES",
  "contactEmail": "info@mumsgardenresort.co.ke",
  "contactPhone": "+254712345678",
  "address": "Kiambu Road, Nairobi",
  "checkInTime": "14:00",
  "checkOutTime": "10:00",
  "isActive": true
}
```

This is the single source of truth for check-in/check-out times shown on the site — never hardcode "2pm" or "14:00-19:00" in frontend copy, always read it from here.

2.2 Room type

```
{
  "id": "a1111111-0000-0000-0000-000000000001",
  "propertyId": "d7f2c9a4-8e31-4b7a-9f2e-222222222222",
  "slug": "deluxe-garden-room",
  "name": "Deluxe Garden Room",
  "description": "Spacious room overlooking the resort gardens...",
  "sizeSqm": 32,
  "bedType": "queen",
  "maxAdults": 2,
  "maxChildren": 1,
  "totalUnits": 8,
  "basePrice": 8500.00,
  "currency": "KES",
  "attributes": {
    "amenities": ["Garden View", "Rain Shower", "TV+VOD", "WiFi", "Mini Fridge"]
  },
  "images": [
    { "id": "img-01", "url": "https://cdn.../deluxe-1.jpg", "isCover": true, "sortOrder": 0 }
  ],
  "isActive": true
}
```

`bedType` is one of: `single`, `twin`, `double`, `queen`, `king`, `bunk`. Always render `attributes.amenities` as a dynamic list — don't assume the same amenity tags exist across properties (Centurion's list will differ from Mum's Garden's).

2.3 Availability result

```
// GET /properties/mums-garden-resort/availability?checkIn=2026-08-10&checkOut=2026-08-12&adults=2
{
  "success": true,
  "data": [
    {
      "roomTypeId": "a1111111-0000-0000-0000-000000000001",
      "name": "Deluxe Garden Room",
      "availableUnits": 3,
      "resolvedPrice": 8500.00,
      "currency": "KES",
      "fitsPartySize": true
    },
    {
      "roomTypeId": "a1111111-0000-0000-0000-000000000002",
      "name": "Standard Room",
      "availableUnits": 0,
      "resolvedPrice": 5500.00,
      "currency": "KES",
      "fitsPartySize": true
    }
  ]
}
```

`availableUnits: 0` means fully booked for those dates — disable the "select" button rather than hiding the room type, so the guest can see it exists and try different dates. `resolvedPrice` already accounts for any active rate-plan override; never recompute pricing client-side from `basePrice`.

2.4 Booking

```
{
  "id": "bk-uuid",
  "bookingCode": "MG-4F82A1",
  "propertyId": "d7f2c9a4-8e31-4b7a-9f2e-222222222222",
  "roomTypeId": "a1111111-0000-0000-0000-000000000001",
  "guestFirstName": "Lawrence",
  "guestLastName": "Otieno",
  "guestEmail": "lawrence@example.com",
  "guestPhone": "+254712345678",
  "checkInDate": "2026-08-10",
  "checkOutDate": "2026-08-12",
  "numAdults": 2,
  "numChildren": 0,
  "numRooms": 1,
  "specialRequests": "Late check-in, arriving ~9pm",
  "status": "pending_payment",
  "subtotal": 17000.00,
  "depositRequired": 5100.00,
  "amountPaid": 0.00,
  "balanceDue": 17000.00,
  "currency": "KES",
  "cancellationDeadlineAt": "2026-07-26T23:59:59Z",
  "expiresAt": "2026-07-23T10:30:00Z"
}
```

`bookingCode` is what you show the guest and what they use to look their booking up later — a human-friendly reference, always prefixed per property (`MG-` for Mum's Garden, `CH-` for Centurion in the examples here — confirm the actual prefixes with backend). `status` is one of: `pending_payment`, `confirmed`, `checked_in`, `checked_out`, `cancelled`, `no_show`, `expired`.

2.5 Payment

```
{
  "id": "pay-uuid",
  "bookingId": "bk-uuid",
  "provider": "mpesa",
  "type": "deposit",
  "amount": 5100.00,
  "currency": "KES",
  "status": "pending",
  "providerReference": "ws_CO_23072026...",
  "mpesaReceiptNumber": null,
  "phoneNumber": "+254712345678",
  "initiatedAt": "2026-07-23T10:15:00Z",
  "completedAt": null
}
```

`status` flow: `initiated` → `pending` → `completed` (or `failed` / `reversed`). The frontend polls for this — see Section 6.

2.6 Offer, gallery item, event type, inquiry

```
// Offer
{
  "id": "...", "title": "Midweek Escape — 20% Off",
  "discountType": "percentage", "discountValue": 20,
  "validFrom": "2026-07-01", "validTo": "2026-08-31",
}
```

```
"imageUrl": "https://cdn.../offer.jpg", "terms": "Valid Mon-Thu, subject to availability." }

// Gallery item
{ "id": "...", "category": "dining", "imageUrl": "https://cdn.../dining-1.jpg",
  "caption": "Sunset dinner on the terrace", "sortOrder": 2 }

// Event type (Meetings & Events content card — NOT a bookable/dated event)
{ "id": "...", "title": "Corporate Board Meetings",
  "description": "Private boardroom with AV setup for up to 20 guests.",
  "imageUrl": "https://cdn.../boardroom.jpg", "icon": "briefcase", "sortOrder": 1 }

// Inquiry (submitted via POST /inquiries)
{ "type": "group_booking", "name": "Jane W.", "email": "jane@example.com",
  "phone": "+254700000000", "message": "Looking to book for a 30-person wedding in October.",
  "partySize": 30, "preferredDate": "2026-10-17" }
```

Event types are editable content cards (private dinners, weddings, corporate training, etc.) — clicking one should route to the contact/inquiry form pre-filled with `type`, not to a booking flow. There is no date/calendar attached to an event type itself.

3. Standard response & error shape

Every endpoint returns one of these two shapes — build one response interceptor/wrapper around this rather than handling each endpoint's shape individually.

```
// success
{ "success": true, "data": { ... }, "meta": { "total": 42, "page": 1, "pageSize": 20 } }

// error
{ "success": false,
  "error": { "code": "BOOKING_NOT_AVAILABLE",
            "message": "No rooms of this type available for the selected dates." } }
```

Error code	When you'll see it
VALIDATION_ERROR	Bad/missing fields in your request body
ROOM_TYPE_NOT_FOUND	Bad/stale room type id
BOOKING_NOT_AVAILABLE	No units left for the selected dates
BOOKING_NOT_FOUND	Bad booking code, or email/phone doesn't match on lookup
BOOKING_EXPIRED	Guest came back to pay after the hold window lapsed — restart the booking
PAYMENT_FAILED	Daraja STK push failed or was cancelled by the guest
CANCELLATION_WINDOW_CLOSED	Guest tried to cancel after the free-cancellation deadline
UNAUTHORIZED	Missing/invalid JWT on an auth-required route
FORBIDDEN_PROPERTY_SCOPE	Admin token tried to touch another property's data

4. Public API

Base path for everything below: `/api/v1`. Property-scoped routes are nested under `/properties/:propertySlug/...` — swap the slug and everything else stays identical between Centurion and Mum's Garden.

4.1 Content & availability

Method	Path	Purpose
GET	/properties/:slug	Property info — contact, check-in/out times, currency
GET	/properties/:slug/room-types	List room types (images, price, amenities)
GET	/properties/:slug/room-types/:roomTypeSlug	Single room type detail
GET	/properties/:slug/availability? checkIn=&checkOut=&adults=&children=	Per-room-type available units + resolved price for the date range
GET	/properties/:slug/offers	Active promotions
GET	/properties/:slug/gallery?category=	Gallery images, filterable by category
GET	/properties/:slug/event-types	Meetings & Events content cards
POST	/inquiries	Submit contact / group booking / events form (body includes <code>propertySlug</code>)

4.2 Booking & payment

Method	Path	Purpose
POST	/bookings	Create a booking → returns <code>bookingCode</code> , <code>status: pending_payment</code>
GET	/bookings/:bookingCode?email=	Guest looks up their own booking (email or phone must match — no public listing)
POST	/bookings/:bookingCode/cancel	Guest-initiated cancellation (blocked once past <code>cancellationDeadlineAt</code>)
POST	/payments/mpesa/stkpush	Triggers the M-Pesa prompt on the guest's phone
GET	/payments/:bookingCode/status	Poll this every 2–3s after STK push until <code>completed</code> / <code>failed</code>

POST `/payments/mpesa/callback` exists too, but it's Daraja → server only — your frontend never calls it directly.

EXAMPLE — CREATE BOOKING

```
POST /api/v1/bookings
{
  "propertySlug": "mums-garden-resort",
  "roomTypeId": "a1111111-0000-0000-0000-000000000001",
  "checkInDate": "2026-08-10",
  "checkOutDate": "2026-08-12",
  "numAdults": 2,
  "numChildren": 0,
  "numRooms": 1,
  "guestFirstName": "Lawrence",
  "guestLastName": "Otieno",
  "guestEmail": "lawrence@example.com",
  "guestPhone": "+254712345678",
  "specialRequests": "Late check-in"
}

// 201 response
{
  "success": true,
  "data": {
    "bookingCode": "MG-4F82A1",
    "status": "pending_payment",
    "subtotal": 17000.00,
    "depositRequired": 5100.00,
    "expiresAt": "2026-07-23T10:30:00Z"
  }
}
```

EXAMPLE — TRIGGER STK PUSH

```
POST /api/v1/payments/mpesa/stkpush
{
  "bookingCode": "MG-4F82A1",
  "phoneNumber": "0712345678",
  "amountType": "deposit"    // "deposit" | "balance" | "full"
}

// response — STK prompt is now on the guest's phone
{ "success": true, "data": { "status": "initiated", "providerReference": "ws_CO_23072026..." } }
```

4.3 Guest account API (optional — guest checkout is always allowed too)

Method	Path	Purpose
POST	/auth/register	Create a guest account
POST	/auth/login	Returns a JWT
GET	/auth/me	Current guest's profile
GET	/auth/me/bookings	Logged-in guest's booking history, across properties

5. Booking + payment flow, step by step

This is the sequence your booking widget should implement, for either property:

1. GET `.../availability?checkIn=&checkOut=&adults=` → show room types + prices
2. Guest picks a room type, fills guest details
3. POST `/bookings` → status: pending_payment
(inventory is now held until expiresAt)
4. POST `/payments/mpesa/stkpush { bookingCode, phoneNumber }` → guest's phone gets the M-Pesa prompt
5. Poll GET `/payments/:bookingCode/status` every 2-3s
 - "completed" → booking flips to "confirmed" server-side → show confirmation screen
 - "failed" → show retry button (re-run step 4)
 - still pending after ~60-90s → show "still waiting" state, keep polling a bit longer
6. If the guest abandons the tab before paying, the booking auto-expires (status → "expired")
once expiresAt passes — inventory is released automatically, no frontend action needed.

Don't build your own "is it paid" logic off the `bookings` object during this flow — always resolve payment completion from `GET /payments/:bookingCode/status`, since that's the one endpoint tied directly to the Daraja callback.

6. Admin panel API

Separate JWT (from `/admin/auth/login`), separate base path: `/api/v1/admin/...` . A Centurion staff/admin token is automatically scoped to Centurion's data only — you never need to pass a property filter yourself for non-superadmin roles; the backend enforces it server-side.

6.1 Auth

Method	Path	Purpose
POST	/admin/auth/login	Returns JWT + role + <code>propertyId</code>
GET	/admin/auth/me	

6.2 Bookings & front desk

Method	Path	Purpose
GET	/admin/bookings?status=&from=&to=&search=	<code>search</code> matches guest name/email/phone/booking code
GET	/admin/bookings/:id	
PATCH	/admin/bookings/:id/status	confirm / check-in / check-out / cancel / no-show
POST	/admin/bookings/:id/assign-room	Assign a specific physical room
POST	/admin/bookings/manual	Walk-in/phone booking entered by staff
GET	/admin/calendar?month=&roomTypeId=	Booking calendar feed
GET	/admin/payments?bookingId=&status=	
POST	/admin/payments/:id/refund	Creates a refund request
PATCH	/admin/refunds/:id/status	Mark processed once actually paid out

6.3 Content management — room types, rooms, pricing

Method	Path	Purpose
GET	/admin/room-types	Includes inactive rows (public list hides them)
POST	/admin/room-types	Create room type
PATCH	/admin/room-types/:id	Partial update
DELETE	/admin/room-types/:id	Soft delete (never removes a room type with existing bookings)
POST	/admin/room-types/:id/images	Multipart image upload
GET / POST	/admin/rooms	Physical room inventory (per room type)
PATCH	/admin/rooms/:id/status	<code>active</code> / <code>maintenance</code> / <code>out_of_service</code>
GET / POST	/admin/rate-plans	Date-bound price/cancellation overrides
GET / POST	/admin/cancellation-policies	Includes <code>tiers</code> array + <code>depositPct</code>
PATCH	/admin/cancellation-policies/:id/set-default	Flips default, unsets the previous one

6.4 Gallery, offers, event-types, inquiries

Method	Path	Purpose
GET / POST	/admin/gallery	Multipart upload with category + caption
PATCH	/admin/gallery/reorder	Bulk drag-and-drop reorder in one call
GET / POST	/admin/event-types	Meetings & Events content cards
GET / POST	/admin/offers	Includes expired/inactive
GET	/admin/inquiries?type=&status=	
PATCH	/admin/inquiries/:id/status	<code>new</code> / <code>contacted</code> / <code>closed</code>

6.5 Property settings, staff, reports, uploads

Method	Path	Purpose
GET /	/admin/property	Contact info, address, check-in/out times — the place that fixes any check-in-time

PATCH		discrepancy
GET / POST	/admin/staff	superadmin only — invite/manage staff & admin accounts
GET	/admin/reports/occupancy?from=&to=	Per-room-type occupancy %
GET	/admin/reports/revenue?from=&to=	Booked vs. collected revenue by room type
POST	/admin/media/upload	Generic file upload used under the hood by room-type images, gallery, offers, event-types

All admin list endpoints share the same conventions: `?page=&pageSize=` (default 20), `?sort=field:asc|desc` , and every list response includes `meta: { total, page, pageSize }` — build one shared pagination component around this.

7. Roles & permissions cheat sheet

Role	Scope	Can do
staff	One property	View/manage bookings, check-in/out, assign rooms, view payments & calendar, manual bookings, respond to inquiries
admin	One property	Everything staff can, plus full CRUD on room types, rooms, rate plans, cancellation policies, offers, gallery, event-types, property settings, refunds
superadmin	All properties (Straight Group ops)	Everything admin can, across every property, plus managing staff/admin accounts

A `staff / admin` token is hard-locked to its own `propertyId` — a Centurion admin literally cannot query Mum's Garden data, even by editing the request. Only `superadmin` tokens can pass `?propertyId=` to cross properties (e.g. for group-wide reports). If you get `FORBIDDEN_PROPERTY_SCOPE` , this is why.

8. Still pending client sign-off

These affect the actual data in the system, not the API shape — don't hardcode fallback values for any of these anywhere in the frontend:

1. Final room specs (size, bed type, occupancy, price) per room type, per property.
2. One official check-in/check-out time per property (already resolvable via `GET /properties/:slug` once set).
3. Cancellation deposit percentage — currently unset. Don't assume a number; render whatever `depositRequired` the booking response actually returns.
4. Card payments are reserved in the schema but not wired up yet — M-Pesa is the only live payment method for now.